Windows in Kubernetes

Kubernetes supports nodes that run Microsoft Windows.

- 1: Windows containers in Kubernetes
- 2: Guide for Running Windows Containers in Kubernetes

Kubernetes supports worker nodes running either Linux or Microsoft Windows.

> ⃠ This item links to a third party project or product that is not part of Kubernetes itself. More information

The CNCF and its parent the Linux Foundation take a vendor-neutral approach towards compatibility. It is possible to join your Windows server as a worker node to a Kubernetes cluster.

You can install and set up kubectl on Windows no matter what operating system you use within your cluster.

If you are using Windows nodes, you can read:

- Networking On Windows
- Windows Storage In Kubernetes
- Resource Management for Windows Nodes
- Configure RunAsUserName for Windows Pods and Containers
- Create A Windows HostProcess Pod
- Configure Group Managed Service Accounts for Windows Pods and Containers
- Security For Windows Nodes
- Windows Debugging Tips
- Guide for Scheduling Windows Containers in Kubernetes

or, for an overview, read:

1 - Windows containers in Kubernetes

Windows applications constitute a large portion of the services and applications that run in many organizations. Windows containers provide a way to encapsulate processes and package dependencies, making it easier to use DevOps practices and follow cloud native patterns for Windows applications.

Organizations with investments in Windows-based applications and Linux-based applications don't have to look for separate orchestrators to manage their workloads, leading to increased operational efficiencies across their deployments, regardless of operating system.

Windows nodes in Kubernetes

To enable the orchestration of Windows containers in Kubernetes, include Windows nodes in your existing Linux cluster. Scheduling Windows containers in Pods on Kubernetes is similar to scheduling Linux-based containers.

In order to run Windows containers, your Kubernetes cluster must include multiple operating systems. While you can only run the control plane on Linux, you can deploy worker nodes running either Windows or Linux.

Windows nodes are supported provided that the operating system is Windows Server 2022 or Windows Server 2025.

This document uses the term *Windows containers* to mean Windows containers with process isolation. Kubernetes does not support running Windows containers with Hyper-V isolation.

Compatibility and limitations

Some node features are only available if you use a specific container runtime; others are not available on Windows nodes, including:

- HugePages: not supported for Windows containers
- Privileged containers: not supported for Windows containers. HostProcess Containers offer similar functionality.
- TerminationGracePeriod: requires containerD

Not all features of shared namespaces are supported. See API compatibility for more details.

See Windows OS version compatibility for details on the Windows versions that Kubernetes is tested against.

From an API and kubectl perspective, Windows containers behave in much the same way as Linux-based containers. However, there are some notable differences in key functionality which are outlined in this section.

Comparison with Linux

Key Kubernetes elements work the same way in Windows as they do in Linux. This section refers to several key workload abstractions and how they map to Windows.

- Pods

  A Pod is the basic building block of Kubernetes–the smallest and simplest unit in the Kubernetes object model that you create or deploy. You may not deploy Windows and Linux containers in the same Pod. All containers in a Pod are scheduled onto a single Node where each Node represents a specific platform and architecture. The following Pod capabilities, properties and events are supported with Windows containers:

  - Single or multiple containers per Pod with process isolation and volume sharing
  - Pod `status` fields
  - Readiness, liveness, and startup probes
  - postStart & preStop container lifecycle hooks
  - ConfigMap, Secrets: as environment variables or volumes
  - `emptyDir` volumes
  - Named pipe host mounts
  - Resource limits
  - OS field:

    The `.spec.os.name` field should be set to `windows` to indicate that the current Pod uses Windows containers.

If you set the `.spec.os.name` field to `windows`, you must not set the following fields in the `.spec` of that Pod:

- `spec.hostPID`
- `spec.hostIPC`
- `spec.securityContext.seLinuxOptions`
- `spec.securityContext.seccompProfile`
- `spec.securityContext.fsGroup`
- `spec.securityContext.fsGroupChangePolicy`
- `spec.securityContext.sysctls`
- `spec.shareProcessNamespace`
- `spec.securityContext.runAsUser`
- `spec.securityContext.runAsGroup`
- `spec.securityContext.supplementalGroups`
- `spec.containers[*].securityContext.seLinuxOptions`
- `spec.containers[*].securityContext.seccompProfile`
- `spec.containers[*].securityContext.capabilities`
- `spec.containers[*].securityContext.readOnlyRootFilesystem`
- `spec.containers[*].securityContext.privileged`
- `spec.containers[*].securityContext.allowPrivilegeEscalation`
- `spec.containers[*].securityContext.procMount`
- `spec.containers[*].securityContext.runAsUser`
- `spec.containers[*].securityContext.runAsGroup`

In the above list, wildcards (`*`) indicate all elements in a list. For example, `spec.containers[*].securityContext` refers to the SecurityContext object for all containers. If any of these fields is specified, the Pod will not be admitted by the API server.

- Workload resources including:
  - ReplicaSet
  - Deployment
  - StatefulSet
  - DaemonSet
  - Job
  - CronJob
  - ReplicationController

- Services See Load balancing and Services for more details.

Pods, workload resources, and Services are critical elements to managing Windows workloads on Kubernetes. However, on their own they are not enough to enable the proper lifecycle management of Windows workloads in a dynamic cloud native environment.

- `kubectl exec`
- Pod and container metrics
- Horizontal pod autoscaling
- Resource quotas
- Scheduler preemption

Command line options for the kubelet

Some kubelet command line options behave differently on Windows, as described below:

- The `--windows-priorityclass` lets you set the scheduling priority of the kubelet process (see CPU resource management)
- The `--kube-reserved`, `--system-reserved`, and `--eviction-hard` flags update NodeAllocatable
- Eviction by using `--enforce-node-allocable` is not implemented
- When running on a Windows node the kubelet does not have memory or CPU restrictions. `--kube-reserved` and `--system-reserved` only subtract from `NodeAllocatable` and do not guarantee resource provided for workloads. See Resource Management for Windows nodes for more information.
- The `PIDPressure` Condition is not implemented
- The kubelet does not take OOM eviction actions

API compatibility

There are subtle differences in the way the Kubernetes APIs work for Windows due to the OS and container runtime. Some workload properties were designed for Linux, and fail to run on Windows.

At a high level, these OS concepts are different:

- Identity - Linux uses userID (UID) and groupID (GID) which are represented as integer types. User and group names are not canonical - they are just an alias in `/etc/groups` or `/etc/passwd` back to UID+GID. Windows uses a larger binary security identifier (SID)

which is stored in the Windows Security Access Manager (SAM) database. This database is not shared between the host and containers, or between containers.

- File permissions - Windows uses an access control list based on (SIDs), whereas POSIX systems such as Linux use a bitmask based on object permissions and UID+GID, plus *optional* access control lists.
- File paths - the convention on Windows is to use `\` instead of `/`. The Go IO libraries typically accept both and just make it work, but when you're setting a path or command line that's interpreted inside a container, `\` may be needed.
- Signals - Windows interactive apps handle termination differently, and can implement one or more of these:
  - A UI thread handles well-defined messages including `WM_CLOSE`.
  - Console apps handle Ctrl-C or Ctrl-break using a Control Handler.
  - Services register a Service Control Handler function that can accept `SERVICE_CONTROL_STOP` control codes.

Container exit codes follow the same convention where 0 is success, and nonzero is failure. The specific error codes may differ across Windows and Linux. However, exit codes passed from the Kubernetes components (kubelet, kube-proxy) are unchanged.

Field compatibility for container specifications

The following list documents differences between how Pod container specifications work between Windows and Linux:

- Huge pages are not implemented in the Windows container runtime, and are not available. They require asserting a user privilege that's not configurable for containers.
- `requests.cpu` and `requests.memory` - requests are subtracted from node available resources, so they can be used to avoid overprovisioning a node. However, they cannot be used to guarantee resources in an overprovisioned node. They should be applied to all containers as a best practice if the operator wants to avoid overprovisioning entirely.
- `securityContext.allowPrivilegeEscalation` - not possible on Windows; none of the capabilities are hooked up
- `securityContext.capabilities` - POSIX capabilities are not implemented on Windows
- `securityContext.privileged` - Windows doesn't support privileged containers, use HostProcess Containers instead
- `securityContext.procMount` - Windows doesn't have a `/proc` filesystem

- `securityContext.readOnlyRootFilesystem` - not possible on Windows; write access is required for registry & system processes to run inside the container
- `securityContext.runAsGroup` - not possible on Windows as there is no GID support
- `securityContext.runAsNonRoot` - this setting will prevent containers from running as `ContainerAdministrator` which is the closest equivalent to a root user on Windows.
- `securityContext.runAsUser` - use `runAsUserName` instead
- `securityContext.seLinuxOptions` - not possible on Windows as SELinux is Linux-specific
- `terminationMessagePath` - this has some limitations in that Windows doesn't support mapping single files. The default value is `/dev/termination-log`, which does work because it does not exist on Windows by default.

Field compatibility for Pod specifications

The following list documents differences between how Pod specifications work between Windows and Linux:

- `hostIPC` and `hostpid` - host namespace sharing is not possible on Windows
- `hostNetwork` - host networking is not possible on Windows
- `dnsPolicy` - setting the Pod `dnsPolicy` to `ClusterFirstWithHostNet` is not supported on Windows because host networking is not provided. Pods always run with a container network.
- `podSecurityContext` see below
- `shareProcessNamespace` - this is a beta feature, and depends on Linux namespaces which are not implemented on Windows. Windows cannot share process namespaces or the container's root filesystem. Only the network can be shared.
- `terminationGracePeriodSeconds` - this is not fully implemented in Docker on Windows, see the GitHub issue. The behavior today is that the ENTRYPOINT process is sent CTRL_SHUTDOWN_EVENT, then Windows waits 5 seconds by default, and finally shuts down all processes using the normal Windows shutdown behavior. The 5 second default is actually in the Windows registry inside the container, so it can be overridden when the container is built.
- `volumeDevices` - this is a beta feature, and is not implemented on Windows. Windows cannot attach raw block devices to pods.
- `volumes`
  - If you define an `emptyDir` volume, you cannot set its volume source to `memory`.
- You cannot enable `mountPropagation` for volume mounts as this is not supported on Windows.

Host network access

Kubernetes v1.26 to v1.32 included alpha support for running Windows Pods in the host's network namespace.

Kubernetes v1.36 does **not** include the `WindowsHostNetwork` feature gate or support for running Windows Pods in the host's network namespace.

Field compatibility for Pod security context

Only the `securityContext.runAsNonRoot` and `securityContext.windowsOptions` from the Pod `securityContext` fields work on Windows.

Node problem detector

The node problem detector (see Monitor Node Health) has preliminary support for Windows. For more information, visit the project's GitHub page.

Pause container

In a Kubernetes Pod, an infrastructure or “pause” container is first created to host the container. In Linux, the cgroups and namespaces that make up a pod need a process to maintain their continued existence; the pause process provides this. Containers that belong to the same pod, including infrastructure and worker containers, share a common network endpoint (same IPv4 and / or IPv6 address, same network port spaces). Kubernetes uses pause containers to allow for worker containers crashing or restarting without losing any of the networking configuration.

Kubernetes maintains a multi-architecture image that includes support for Windows. For Kubernetes v1.36.0 the recommended pause image is `registry.k8s.io/pause:3.6` . The source code is available on GitHub.

Microsoft maintains a different multi-architecture image, with Linux and Windows amd64 support, that you can find as `mcr.microsoft.com/oss/kubernetes/pause:3.6` . This image is built from the same source as the Kubernetes maintained image but all of the Windows binaries are authenticode signed by Microsoft. The Kubernetes project recommends using the Microsoft maintained image if you are deploying to a production or production-like environment that requires signed binaries.

Container runtimes

You need to install a container runtime into each node in the cluster so that Pods can run there.

The following container runtimes work with Windows:

> **Note:** This section links to third party projects that provide functionality required by Kubernetes. The Kubernetes project authors aren't responsible for these projects, which are listed alphabetically. To add a project to this list, read the content guide before submitting a change. More information.

ContainerD

FEATURE STATE: Kubernetes v1.20 [stable]

You can use ContainerD 1.4.0+ as the container runtime for Kubernetes nodes that run Windows.

Learn how to install ContainerD on a Windows node.

> **Note:**
> There is a known limitation when using GMSA with containerd to access Windows network shares, which requires a kernel patch.

Mirantis Container Runtime

Mirantis Container Runtime (MCR) is available as a container runtime for all Windows Server 2019 and later versions.

See Install MCR on Windows Servers for more information.

Windows OS version compatibility

On Windows nodes, strict compatibility rules apply where the host OS version must match the container base image OS version.

For Kubernetes v1.36, operating system compatibility for Windows nodes (and Pods) is as follows:

Windows Server LTSC release

Windows Server 2022

Windows Server 2025

The Kubernetes version-skew policy also applies.

Hardware recommendations and considerations

> **Note:** This section links to third party projects that provide functionality required by Kubernetes. The Kubernetes project authors aren't responsible for these projects, which are listed alphabetically. To add a project to this list, read the content guide before submitting a change. More information.

> **Note:**
> The following hardware specifications outlined here should be regarded as sensible default values. They are not intended to represent minimum requirements or specific recommendations for production environments. Depending on the requirements for your workload these values may need to be adjusted.

- 64-bit processor 4 CPU cores or more, capable of supporting virtualization
- 8GB or more of RAM
- 50GB or more of free disk space

Refer to Hardware requirements for Windows Server Microsoft documentation for the most up-to-date information on minimum hardware requirements. For guidance on deciding on resources for production worker nodes refer to Production worker nodes Kubernetes documentation.

To optimize system resources, if a graphical user interface is not required, it may be preferable to use a Windows Server OS installation that excludes the Windows Desktop Experience installation option, as this configuration typically frees up more system resources.

In assessing disk space for Windows worker nodes, take note that Windows container images are typically larger than Linux container images, with container image sizes ranging from 300MB to over 10GB for a single image. Additionally, take note that the `c:` drive in Windows containers represents a virtual free size of 20GB by default, which is not the actual consumed space, but rather the disk size for which a single container can grow to occupy when using local storage on the host. See Containers on Windows - Container Storage Documentation for more detail.

Getting help and troubleshooting

Your main source of help for troubleshooting your Kubernetes cluster should start with the Troubleshooting page.

Some additional, Windows-specific troubleshooting help is included in this section. Logs are an important element of troubleshooting issues in Kubernetes. Make sure to include them any time you seek troubleshooting assistance from other contributors. Follow the instructions in the SIG Windows contributing guide on gathering logs.

Reporting issues and feature requests

If you have what looks like a bug, or you would like to make a feature request, please follow the SIG Windows contributing guide to create a new issue. You should first search the list of issues in case it was reported previously and comment with your experience on the issue and add additional logs. SIG Windows channel on the Kubernetes Slack is also a great avenue to get some initial support and troubleshooting ideas prior to creating a ticket.

Validating the Windows cluster operability

The Kubernetes project provides a *Windows Operational Readiness* specification, accompanied by a structured test suite. This suite is split into two sets of tests, core and extended, each containing categories aimed at testing specific areas. It can be used to validate all the functionalities of a Windows and hybrid system (mixed with Linux nodes) with full coverage.

To set up the project on a newly created cluster, refer to the instructions in the project guide.

Deployment tools

The kubeadm tool helps you to deploy a Kubernetes cluster, providing the control plane to manage the cluster it, and nodes to run your workloads.

The Kubernetes cluster API project also provides means to automate deployment of Windows nodes.

Windows distribution channels

For a detailed explanation of Windows distribution channels see the Microsoft documentation.

Information on the different Windows Server servicing channels including their support models can be found at Windows Server servicing channels.

2 - Guide for Running Windows Containers in Kubernetes

This page provides a walkthrough for some steps you can follow to run Windows containers using Kubernetes. The page also highlights some Windows specific functionality within Kubernetes.

It is important to note that creating and deploying services and workloads on Kubernetes behaves in much the same way for Linux and Windows containers. The kubectl commands to interface with the cluster are identical. The examples in this page are provided to jumpstart your experience with Windows containers.

Objectives

Configure an example deployment to run Windows containers on a Windows node.

Before you begin

You should already have access to a Kubernetes cluster that includes a worker node running Windows Server.

Getting Started: Deploying a Windows workload

The example YAML file below deploys a simple webserver application running inside a Windows container.

Create a manifest named `win-webserver.yaml` with the contents below:

```
---
apiVersion: v1
kind: Service
metadata:
  name: win-webserver
  labels:
    app: win-webserver
spec:
  ports:
```

```
    # the port that this service should serve on
    - port: 80
      targetPort: 80
  selector:
    app: win-webserver
  type: NodePort
---
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: win-webserver
  name: win-webserver
spec:
  replicas: 2
  selector:
    matchLabels:
      app: win-webserver
  template:
    metadata:
      labels:
        app: win-webserver
      name: win-webserver
    spec:
     containers:
      - name: windowswebserver
        image: mcr.microsoft.com/windows/servercore:ltsc2019
        command:
        - powershell.exe
        - -command
        - "<#code used from https://gist.github.com/19WAS85/5424431#> ; $$listener =
     nodeSelector:
      kubernetes.io/os: windows
```

> **Note:**
> Port mapping is also supported, but for simplicity this example exposes port 80 of the container directly to the Service.

1. Check that all nodes are healthy:

```
kubectl get nodes
```

2. Deploy the service and watch for pod updates:

```
kubectl apply -f win-webserver.yaml
kubectl get pods -o wide -w
```

When the service is deployed correctly both Pods are marked as Ready. To exit the watch command, press Ctrl+C.

3. Check that the deployment succeeded. To verify:

   - Several pods listed from the Linux control plane node, use `kubectl get pods`
   - Node-to-pod communication across the network, `curl` port 80 of your pod IPs from the Linux control plane node to check for a web server response
   - Pod-to-pod communication, ping between pods (and across hosts, if you have more than one Windows node) using `kubectl exec`
   - Service-to-pod communication, `curl` the virtual service IP (seen under `kubectl get services`) from the Linux control plane node and from individual pods
   - Service discovery, `curl` the service name with the Kubernetes default DNS suffix
   - Inbound connectivity, `curl` the NodePort from the Linux control plane node or machines outside of the cluster
   - Outbound connectivity, `curl` external IPs from inside the pod using `kubectl exec`

> **Note:**
> Windows container hosts are not able to access the IP of services scheduled on them due to current platform limitations of the Windows networking stack. Only Windows pods are able to access service IPs.

Observability

Capturing logs from workloads

Logs are an important element of observability; they enable users to gain insights into the operational aspect of workloads and are a key ingredient to troubleshooting issues. Because Windows containers and workloads inside Windows containers behave differently from Linux containers, users had a hard time collecting logs, limiting operational visibility. Windows workloads for example are usually configured to log to ETW (Event Tracing for Windows) or push entries to the application event log. LogMonitor, an open source tool by Microsoft, is the recommended way to monitor configured log sources inside a Windows container. LogMonitor supports monitoring event logs, ETW providers, and custom application logs, piping them to STDOUT for consumption by `kubectl logs <pod>`.

Follow the instructions in the LogMonitor GitHub page to copy its binaries and configuration files to all your containers and add the necessary entrypoints for LogMonitor to push your logs to STDOUT.

Configuring container user

Using configurable Container usernames

Windows containers can be configured to run their entrypoints and processes with different usernames than the image defaults. Learn more about it here.

Managing Workload Identity with Group Managed Service Accounts

Windows container workloads can be configured to use Group Managed Service Accounts (GMSA). Group Managed Service Accounts are a specific type of Active Directory account that provide automatic password management, simplified service principal name (SPN) management, and the ability to delegate the management to other administrators across multiple servers. Containers configured with a GMSA can access external Active Directory Domain resources while carrying the identity configured with the GMSA. Learn more about configuring and using GMSA for Windows containers here.

Taints and tolerations

Users need to use some combination of taint and node selectors in order to schedule Linux and Windows workloads to their respective OS-specific nodes. The recommended

approach is outlined below, with one of its main goals being that this approach should not break compatibility for existing Linux workloads.

You can (and should) set `.spec.os.name` for each Pod, to indicate the operating system that the containers in that Pod are designed for. For Pods that run Linux containers, set `.spec.os.name` to `linux`. For Pods that run Windows containers, set `.spec.os.name` to `windows`.

> **Note:**
> If you are running a version of Kubernetes older than 1.24, you may need to enable the `IdentifyPodOS` feature gate to be able to set a value for `.spec.pod.os`.

The scheduler does not use the value of `.spec.os.name` when assigning Pods to nodes. You should use normal Kubernetes mechanisms for assigning pods to nodes to ensure that the control plane for your cluster places pods onto nodes that are running the appropriate operating system.

The `.spec.os.name` value has no effect on the scheduling of the Windows pods, so taints and tolerations (or node selectors) are still required to ensure that the Windows pods land onto appropriate Windows nodes.

Ensuring OS-specific workloads land on the appropriate container host

Users can ensure Windows containers can be scheduled on the appropriate host using taints and tolerations. All Kubernetes nodes running Kubernetes 1.36 have the following default labels:

- kubernetes.io/os = [windows|linux]
- kubernetes.io/arch = [amd64|arm64|...]

If a Pod specification does not specify a `nodeSelector` such as `"kubernetes.io/os": windows`, it is possible the Pod can be scheduled on any host, Windows or Linux. This can be problematic since a Windows container can only run on Windows and a Linux container can only run on Linux. The best practice for Kubernetes 1.36 is to use a `nodeSelector`.

However, in many cases users have a pre-existing large number of deployments for Linux containers, as well as an ecosystem of off-the-shelf configurations, such as community Helm charts, and programmatic Pod generation cases, such as with operators. In those

situations, you may be hesitant to make the configuration change to add `nodeSelector` fields to all Pods and Pod templates. The alternative is to use taints. Because the kubelet can set taints during registration, it could easily be modified to automatically add a taint when running on Windows only.

For example: `--register-with-taints='os=windows:NoSchedule'`

By adding a taint to all Windows nodes, nothing will be scheduled on them (that includes existing Linux Pods). In order for a Windows Pod to be scheduled on a Windows node, it would need both the `nodeSelector` and the appropriate matching toleration to choose Windows.

```
nodeSelector:
    kubernetes.io/os: windows
    node.kubernetes.io/windows-build: '10.0.20348'
tolerations:
    - key: "os"
      operator: "Equal"
      value: "windows"
      effect: "NoSchedule"
```

Handling multiple Windows versions in the same cluster

The Windows Server version used by each pod must match that of the node. If you want to use multiple Windows Server versions in the same cluster, then you should set additional node labels and `nodeSelector` fields.

Kubernetes automatically adds a label, `node.kubernetes.io/windows-build` to simplify this.

This label reflects the Windows major, minor, and build number that need to match for compatibility. Here are values used for each Windows Server version:

Product Name	Version
Windows Server 2022	10.0.20348
Windows Server 2025	10.0.26100

Simplifying with RuntimeClass

RuntimeClass can be used to simplify the process of using taints and tolerations. A cluster administrator can create a `RuntimeClass` object which is used to encapsulate these taints and tolerations.

1. Save this file to `runtimeClasses.yml`. It includes the appropriate `nodeSelector` for the Windows OS, architecture, and version.

```
---
apiVersion: node.k8s.io/v1
kind: RuntimeClass
metadata:
  name: windows-2019
handler: example-container-runtime-handler
scheduling:
  nodeSelector:
    kubernetes.io/os: 'windows'
    kubernetes.io/arch: 'amd64'
    node.kubernetes.io/windows-build: '10.0.20348'
  tolerations:
  - effect: NoSchedule
    key: os
    operator: Equal
    value: "windows"
```

2. Run `kubectl create -f runtimeClasses.yml` using as a cluster administrator
3. Add `runtimeClassName: windows-2019` as appropriate to Pod specs

   For example:

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: iis-2019
  labels:
    app: iis-2019
spec:
  replicas: 1
  template:
```

```
    metadata:
      name: iis-2019
      labels:
        app: iis-2019
    spec:
      runtimeClassName: windows-2019
      containers:
      - name: iis
        image: mcr.microsoft.com/windows/servercore/iis:windowsservercore-ltsc201
        resources:
          limits:
            cpu: 1
            memory: 800Mi
          requests:
            cpu: .1
            memory: 300Mi
        ports:
          - containerPort: 80
 selector:
    matchLabels:
      app: iis-2019
---
apiVersion: v1
kind: Service
metadata:
  name: iis
spec:
  type: LoadBalancer
  ports:
  - protocol: TCP
    port: 80
  selector:
    app: iis-2019
```